Algoritmos

Ignacio Don

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a mathlab y trabaja con variables que en general pueden ser matrices.

Uso de datasets o bases de datos internos de R

Para ver una columna especifica de un "dataset" puedo usar el signo "$", entonces yo por ejemplo si quiero ver la velocidad de la lista "cars" puedo hacer "cars$speed". Tambien puedo usar el comando data() si quiero ver una lista de distintas bases de datos. Si simplemente quiero ver el dataset entero ingreso el nombre de dicho dataset en la consola

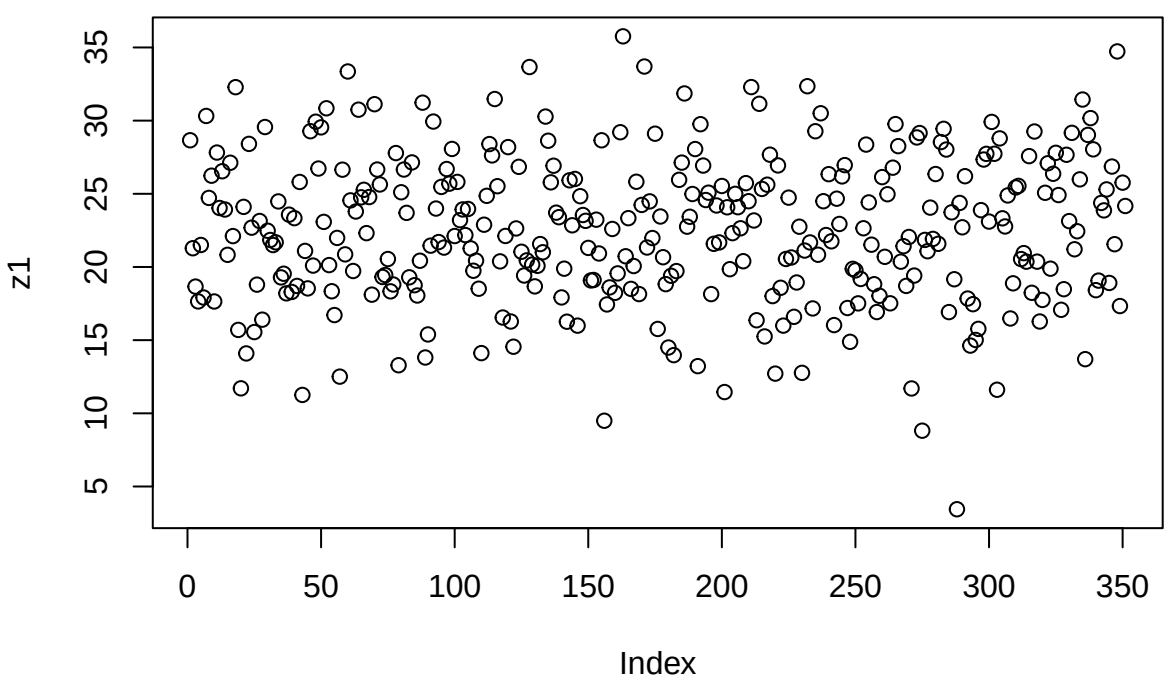
Funciones estadisticas

```
z1<-rnorm(351,22,5)
z1
```

```
##   [1] 28.659467 21.280075 18.659290 17.644962 21.502410 17.913434 30.318013
##   [8] 24.719106 26.237829 17.641619 27.818820 24.031298 26.541845 23.925937
##  [15] 20.830689 27.123279 22.110915 32.286113 15.700442 11.707090 24.107125
##  [22] 14.093009 28.410829 22.686037 15.552008 18.794354 23.141952 16.400233
##  [29] 29.562807 22.475283 21.844206 21.498263 21.686106 24.478981 19.287929
##  [36] 19.538515 18.197165 23.574203 18.291885 23.337059 18.703933 25.808694
##  [43] 11.257750 21.097105 18.536994 29.276808 20.093932 29.928008 26.725880
##  [50] 29.539839 23.081641 30.842925 20.128482 18.342170 16.714419 21.985276
##  [57] 12.507096 26.648267 20.862705 33.358221 24.549961 19.725188 23.790807
##  [64] 30.748791 24.758707 25.274258 22.312542 24.778574 18.108074 31.123385
##  [71] 26.658576 25.637969 19.317346 19.449763 20.543615 18.334299 18.803330
##  [78] 27.784409 13.285932 25.106441 26.658057 23.698041 19.292609 27.144034
##  [85] 18.755834 18.049919 20.425789 31.227767 13.814193 15.390653 21.458222
##  [92] 29.936984 23.992232 21.702520 25.469536 21.325574 26.692005 25.692727
##  [99] 28.065186 22.112762 25.813065 23.200644 23.931348 22.179250 23.962290
## [106] 21.269999 19.742507 20.434023 18.515400 14.114188 22.888296 24.862312
## [113] 28.397979 27.623717 31.475106 25.520112 20.382412 16.544611 22.118969
## [120] 28.190902 16.273939 14.551366 22.632768 26.841783 21.039659 19.416928
## [127] 20.452230 33.661710 20.162860 18.674302 20.080230 21.566037 21.005229
## [134] 30.276361 28.626272 25.783398 26.918165 23.710999 23.401059 17.919096
```

```
## [141] 19.884102 16.260946 25.921713 22.847581 26.014566 15.990418 24.836739
## [148] 23.551745 23.151460 21.303441 19.079767 19.110578 23.233192 20.922735
## [155] 28.656236  9.492598 17.439212 18.603736 22.597171 18.234917 19.561806
## [162] 29.207163 35.759310 20.731068 23.335861 18.502604 20.064076 25.823652
## [169] 18.148346 24.239560 33.697516 21.330003 24.482505 21.976784 29.106697
## [176] 15.764703 23.449735 20.667404 18.826591 14.496668 19.397112 13.975045
## [183] 19.719324 25.955117 27.132219 31.857061 22.755331 23.431534 24.985610
## [190] 28.056737 13.222533 29.756064 26.929592 24.573856 25.076250 18.146772
## [197] 21.583398 24.201490 21.678609 25.534036 11.453117 24.078057 19.848034
## [204] 22.312427 24.996495 24.085233 22.644119 20.393377 25.729510 24.492705
## [211] 32.291770 23.177456 16.360104 31.149506 25.333375 15.252381 25.622420
## [218] 27.672277 18.010537 12.712311 26.939165 18.579334 15.992944 20.546607
## [225] 24.737009 20.648197 16.593553 18.939951 22.750863 12.765261 21.122851
## [232] 32.350417 21.675795 17.165909 29.274110 20.836141 30.502903 24.493043
## [239] 22.183378 26.340536 21.753446 16.029438 24.665883 22.939341 26.190430
## [246] 26.960431 17.195512 14.884395 19.872427 19.766082 17.512028 19.180527
## [253] 22.646388 28.363896 24.410631 21.523288 18.820104 16.920735 18.031474
## [260] 26.139764 20.693378 24.970231 17.515438 26.785351 29.763119 28.255609
## [267] 20.361773 21.419684 18.706206 22.049498 11.698649 19.412531 28.849395
## [274] 29.152098  8.812947 21.850059 21.076624 24.055263 21.918662 26.349830
## [281] 21.584281 28.534312 29.440087 28.025300 16.916238 23.734376 19.160909
## [288]  3.446122 24.375740 22.710583 26.202431 17.842385 14.629095 17.457938
## [295] 15.011764 15.771172 23.883100 27.340577 27.730040 23.102991 29.911994
## [302] 27.734781 11.610383 28.786625 23.324884 22.768419 24.887740 16.480093
## [309] 18.882137 25.433782 25.539762 20.538587 20.948133 20.363073 27.575207
## [316] 18.235693 29.256976 20.364785 16.272740 17.747450 25.069333 27.086532
## [323] 19.883987 26.367805 27.801467 24.913633 17.072802 18.481642 27.666764
## [330] 23.134724 29.165743 21.210623 22.440486 25.988497 31.441301 13.701456
## [337] 29.022740 30.171962 28.037350 18.414097 19.069563 24.375740 23.865853
## [344] 25.300975 18.908130 26.862788 21.555429 34.728921 17.329774 25.762997
## [351] 24.167500
```

```r
plot(z1)
```



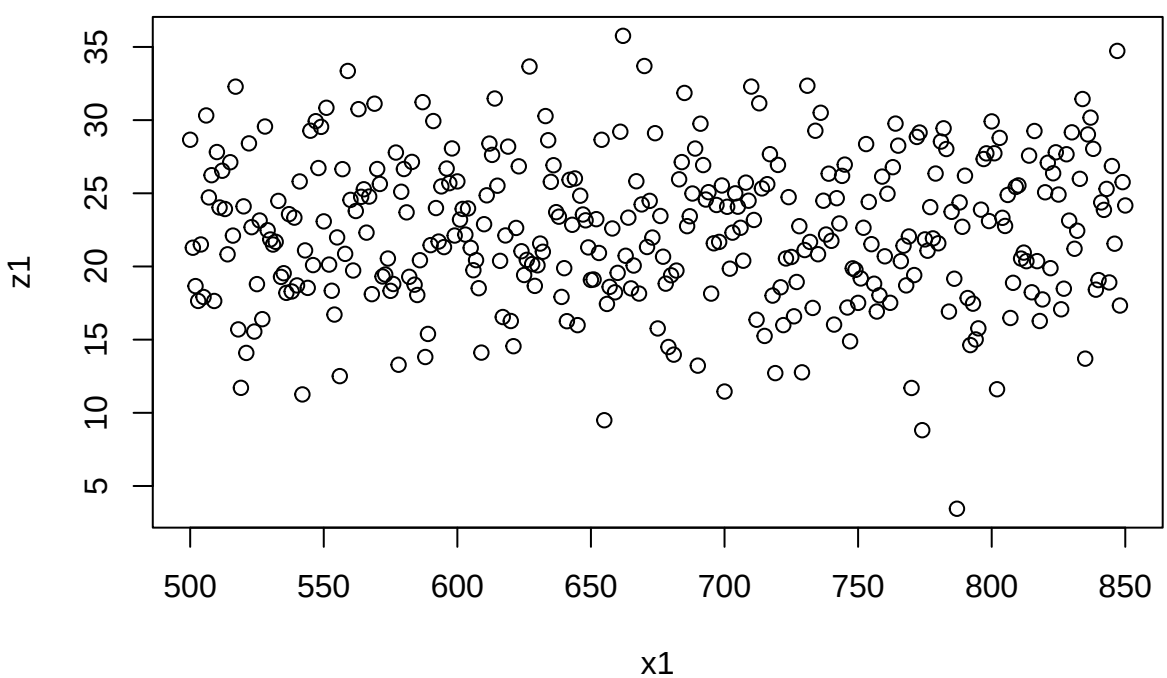
```r
w1<-length(z1)
w1
```

```
## [1] 351
```

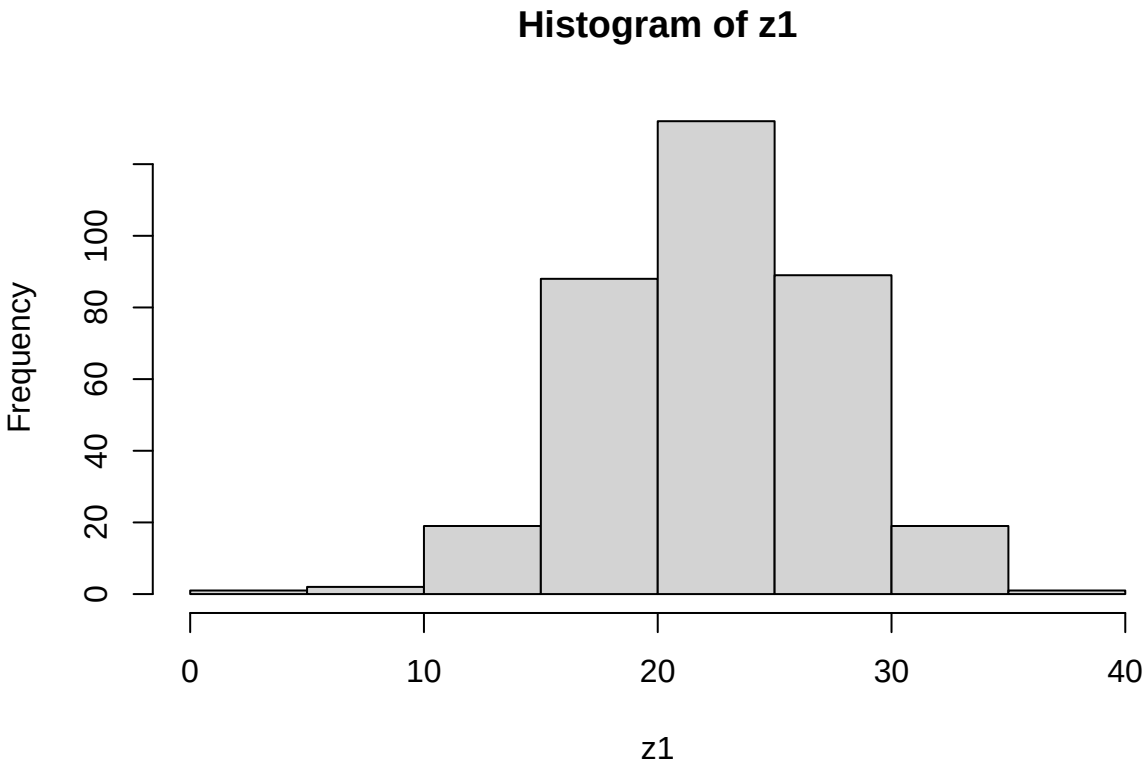
```r
x1<-(500:850)
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849 850
```

```r
plot(x1,z1)
```



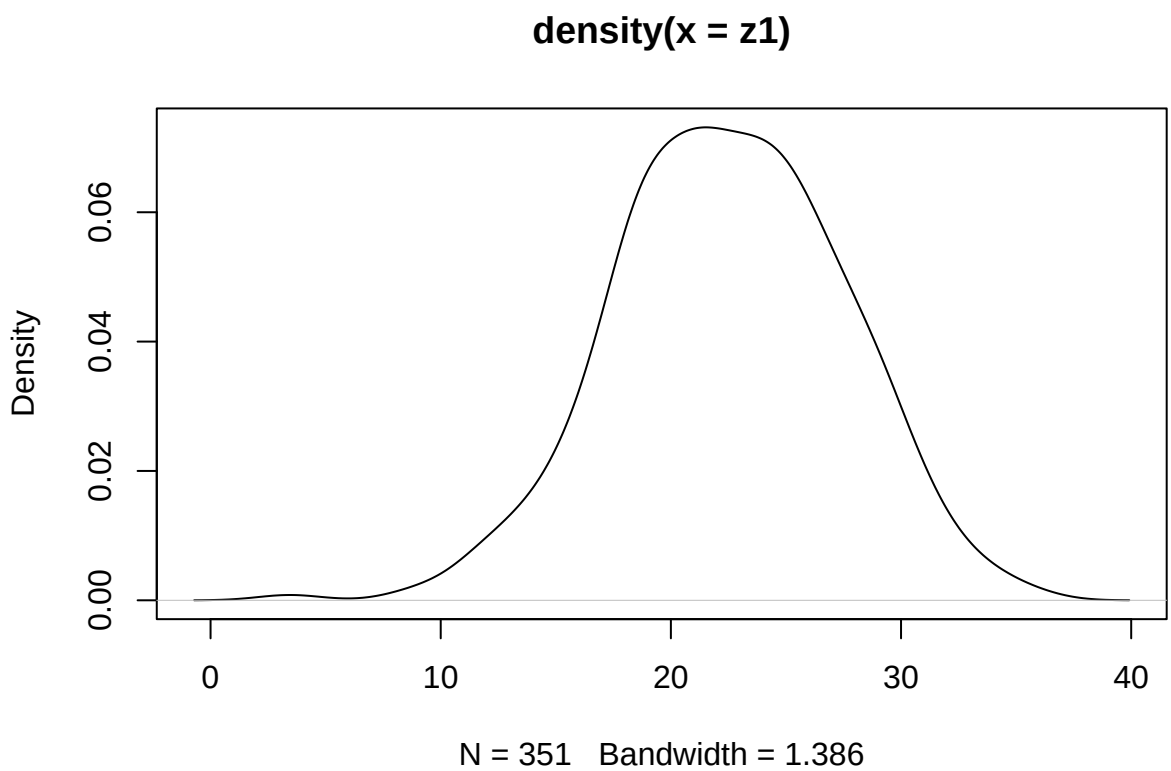
```r
hist(z1)
```



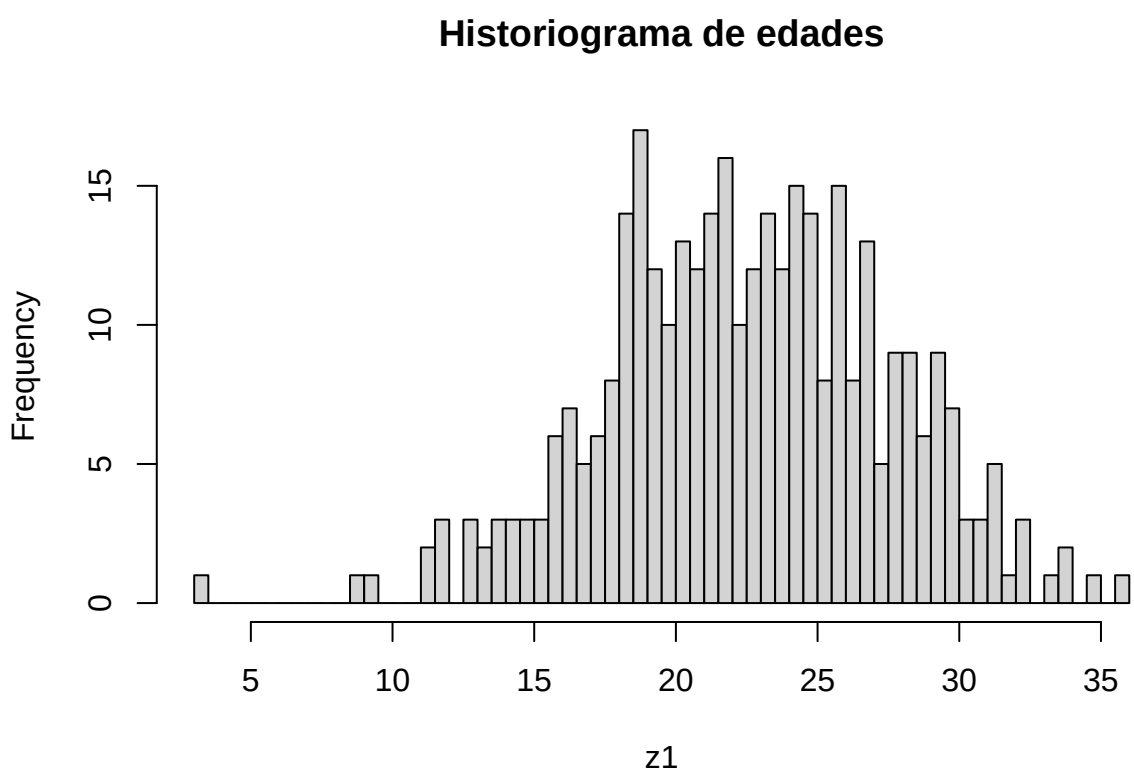

```r
density(z1)
```

```
##
## Call:
##  density.default(x = z1)
##
## Data: z1 (351 obs.); Bandwidth 'bw' = 1.386
##
##        x                 y
##  Min.   :-0.7116   Min.   :9.168e-06
##  1st Qu.: 9.4455   1st Qu.:7.898e-04
##  Median :19.6027   Median :1.069e-02
##  Mean   :19.6027   Mean   :2.456e-02
##  3rd Qu.:29.7599   3rd Qu.:4.984e-02
##  Max.   :39.9171   Max.   :7.313e-02
```

```r
plot(density(z1),type="l")
```




```r
hist(z1,main="Historiograma de edades",breaks=50)
```




```r
A=38
A=40
B=60
C=B-A
```

```r
C
```

```
## [1] 20
```

```r
Y=40
alpha<-30
```

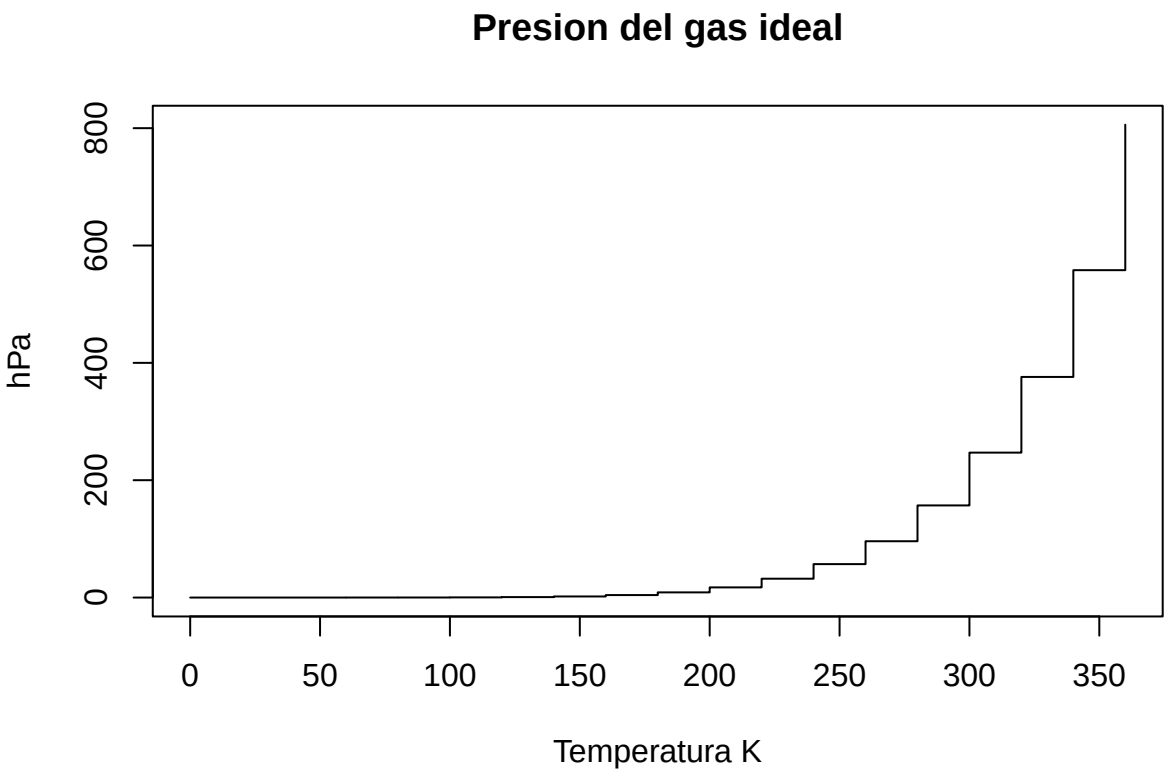
```r
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

Including Plots

You can also embed plots, for example:

```r
plot(pressure,main="Presion del gas ideal",ylab="hPa",xlab="Temperatura K",type="s")
```



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

Las ultimas 3 cifras de mi DNI 550 crear una variavle que tenga ese numero

```r
DNI=550
```

Consigna: crear un vector que tenga todos los numeros del 1 al 550 enteros.

```r
SecuenciaDNI<-(1:550)
```

Ejercicio 2

Calcular la suma de los valores del vector Secuencia_dni

```r
total<-0
valorfinal<-length(SecuenciaDNI)
for (i in 1:valorfinal){
total<-total+SecuenciaDNI[i]
}
print(total)
```

```
## [1] 151525
```

Calcula la suma de todos los valores del vector secuenciaDNI con python.

```r
SecuenciaDNI=range(1,550)
total=sum(SecuenciaDNI)
print(total)
```

```
## 150975
```

Consigna: Calcula la suma de todos los valores del vector secuenciaDNI con Julia.

Cuanto tarda en corrFer el codigo del ejercicio 2

```r
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```

```
##    user  system elapsed
##   0.011   0.000   0.012
```

```r
require(stats)
system.time(for(i in 1:length(SecuenciaDNI)){})
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

```r
total<-0
valorfinal<-length(SecuenciaDNI)
for (i in 1:valorfinal){
total<-total+SecuenciaDNI[i]
}
print(total)
```

```
## [1] 151525
```

```r
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```

```
## Time difference of 14.01282 secs
```

Consigna 7: Repetir el ejercicio anterior usando la cabeza anterior.

Biblioteca tictoc

Esto de usar una biblioteca es llamar u cargar una proced imientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar el comando install.packages() o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones tic y toc son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluacion comparativa que el tiempo de sistema recién demostrado. Sin embargo, tictoc agrega mucha más comodidad al usuario y armonía al conjunto.

```
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
```

```
## [1] "dormire una siestita..."
```

```
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
```

```
## [1] "...suena el despertador"
```

```
## [1] "...suena el despertador"
toc()
```

```
## sleeping: 2.005 sec elapsed
```

Biblioteca rbenchmark

La documentación de la función benchmark del pa quete rbenchmark R lo describe como un simple contenedor alrededor de sys tem.time. Sin embargo, agrega mucha conveniencia en comparación con las lla madas simples a system.time. Por ejemplo, requiere solo una llamada de referencia para cronometrar múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos. ## Implementación de una serie Fibonachi o Fibonacci En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infita de números naturales: 0,1,1,2,3,5,8 ... 89,144,233 ... La sucesión comienza con los números 0 y 1,2 a partir de estos, cada término es la suma de los dos anteriores , es la relación de recurrencia que la de ne. A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. También aparece en con guraciones biológicas, como por ejemplo en las ramas de los árboles, en la disposición de las hojas en el tallo, en las ores de alcachofas y girasoles, en las in orescencias del brécol romanesco, en la con guración de las piñas de las coníferas, en la reproducción de los conejos y en como el ADN codi ca el crecimiento de formas orgánicas complejas. De igual manera, se encuentra en la estructura espiral del caparazón de algunos moluscos, como el nautilus. Original de la Biblioteca Uninersidad de Florencia. Liber Abachi- Autor Fibonacci ¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000 ?

```
# Inicialización
a <- 0
b <- 1
contador <- 1 # ya tenemos el primer número
# Iterar hasta superar 1.000.000
while (b <= 1000000) {
 siguiente <- a + b
 a <- b
 b <- siguiente
 contador <- contador + 1
}
```

```r
# Resultados
cat("Número de iteraciones:", contador, "\n")
```

```
## Número de iteraciones: 31
```

```r
cat("Primer Fibonacci mayor a 1.000.000:", b, "\n")
```

```
## Primer Fibonacci mayor a 1.000.000: 1346269
```

CONSIGNA: Compara la performance de ordenación del método burbuja vs el método sort de R Usar método microbenchmark para una muestra de tamaño 20.000

```r
# Instalar paquete si no lo tenés
if (!require(microbenchmark)) {
 install.packages("microbenchmark")
 library(microbenchmark)
}
```

```
## Loading required package: microbenchmark
```

```r
# Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
# Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
 n <- length(x)
 for (i in 1:(n-1)) {
 for (j in 1:(n-i)) {
 if (x[j] > x[j+1]) {
 temp <- x[j]
 x[j] <- x[j+1]
x[j+1] <- temp
 }
 }
 }
 return(x)
}
# Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
 burbuja = bubble_sort(vector),
 sort_R = sort(vector),
 times = 5
)
print(resultado)
```

```
## Unit: microseconds
##     expr         min           lq         mean      median          uq
##  burbuja 20705367.26 20821552.234 2.081994e+07 20825716.39 20840714.24
##   sort_R      588.27      596.531 7.477604e+02      708.57      784.84
##           max neval
##  20906339.373     5
##      1060.591     5
```

La penitencia de Newton

Comentan algunos historiadores que Newton tenía en la escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto improcedente lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero. **Consigna 10:** Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1*1010 y verifica cuál de los dos es más eficiente.

```r
#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
 suma1 <- suma1 + i
}
suma1
```

```
## [1] 510555
```

```r
#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010
suma2 <- n * (n + 1) / 2
suma2
```

```
## [1] 510555
```

```r
#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
# Método 1
system.time({
 suma1 <- 0
 for (i in 1:1010){
 suma1 <- suma1 + i
 }
})
```

```
##    user  system elapsed
##   0.002   0.000   0.002
```

```r
# Método 2
system.time({
 suma2 <- 1010 * (1010 + 1) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1*1010. El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente.

##Matriz Inversa y Gráficos Violín En este apartado se realiza una comparación del rendimiento computacional de distintos métodos para estimar los coeficientes de un modelo de regresión lineal en R. Para ello, se emplea la librería microbenchmark, la cual permite medir con precisión los tiempos de ejecución de diferentes algoritmos a través de múltiples repeticiones. Se genera un conjunto de datos simulado utilizando funciones

de generación aleatoria, compuesto por una matriz de variables explicativas X de dimensión 10000×100 y un vector respuesta y, construido como una combinación lineal de X con un término de error. Esto permite trabajar en un contexto controlado pero representativo de problemas de gran escala. A continuación, se implementan tres enfoques para el cálculo de los coeficientes: el método clásico mediante la función lm(), el cálculo explícito usando la inversa de la matriz y la resolución directa del sistema lineal equivalente mediante la función solve(). Para asegurar la validez de la comparación, se define una función que verifica que los coeficientes obtenidos por los distintos métodos sean numéricamente equivalentes dentro de una tolerancia establecida. Finalmente, se visualizan los resultados utilizando la librería ggplot2, a través de la función autoplot(), que permite representar gráficamente la distribución de los tiempos de ejecución de cada método, facilitando su análisis comparativo.

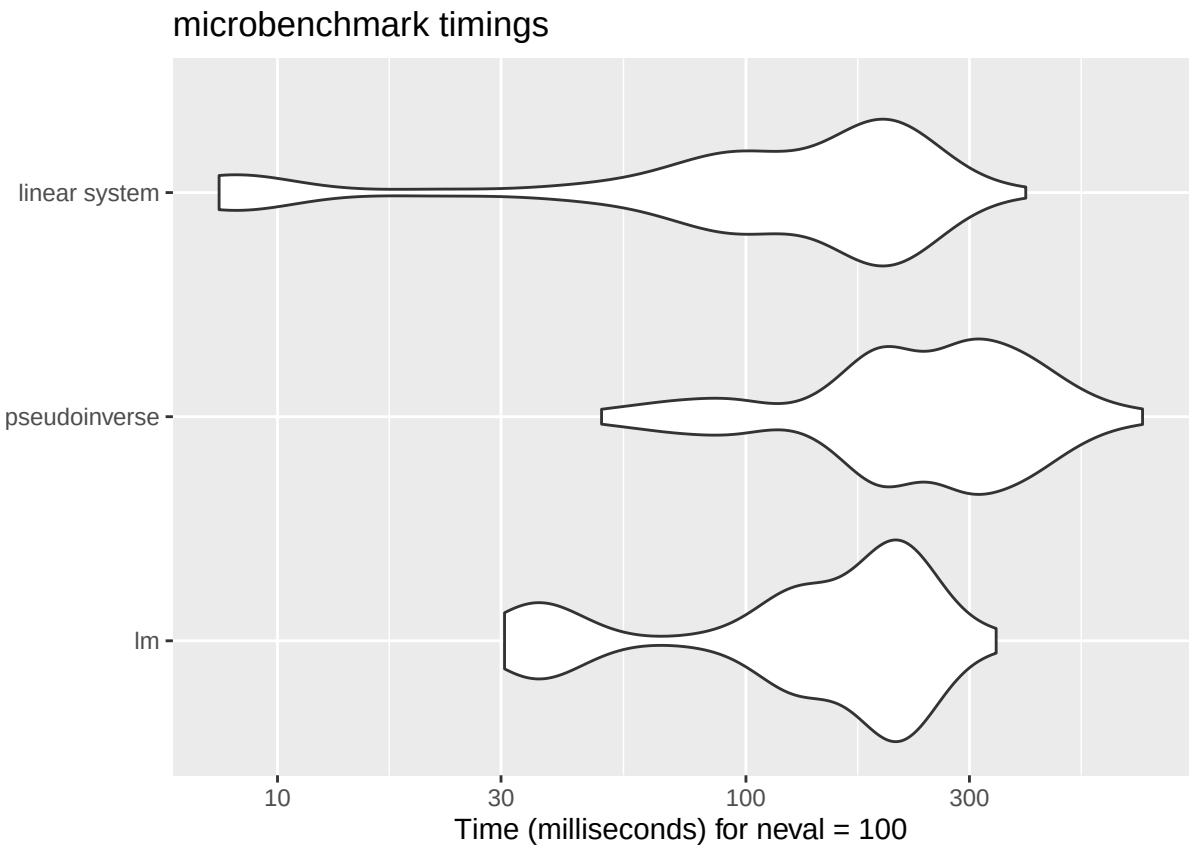
```r
library(microbenchmark)
set.seed(2017)
n <- 10000
p <- 100
X <- matrix(rnorm(n * p), n, p)
y <- X %*% rnorm(p) + rnorm(100)
check_for_equal_coefs <- function(values) {
tol <- 1e-12
max_error <- max(
c(
abs(values[[1]] - values[[2]]),
abs(values[[2]] - values[[3]]),
abs(values[[1]] - values[[3]])
)
)
max_error < tol
}
mbm <- microbenchmark(
"lm" = {
b <- lm(y ~ X + 0)$coef
},
"pseudoinverse" = {
b <- solve(t(X) %*% X) %*% t(X) %*% y
},
"linear system" = {
b <- solve(t(X) %*% X, t(X) %*% y)
},
check = check_for_equal_coefs
)
mbm
```

```
## Unit: milliseconds
##           expr       min        lq     mean   median       uq      max neval
##             lm 30.524394 105.04907 155.1692 182.6424 213.3339 342.0816   100
##  pseudoinverse 49.178139 193.17354 270.0691 271.7595 368.3632 702.5037   100
##  linear system  7.502796  74.34082 134.4913 135.7091 200.2456 395.6828   100
```

```r
library(ggplot2)
autoplot(mbm)
```

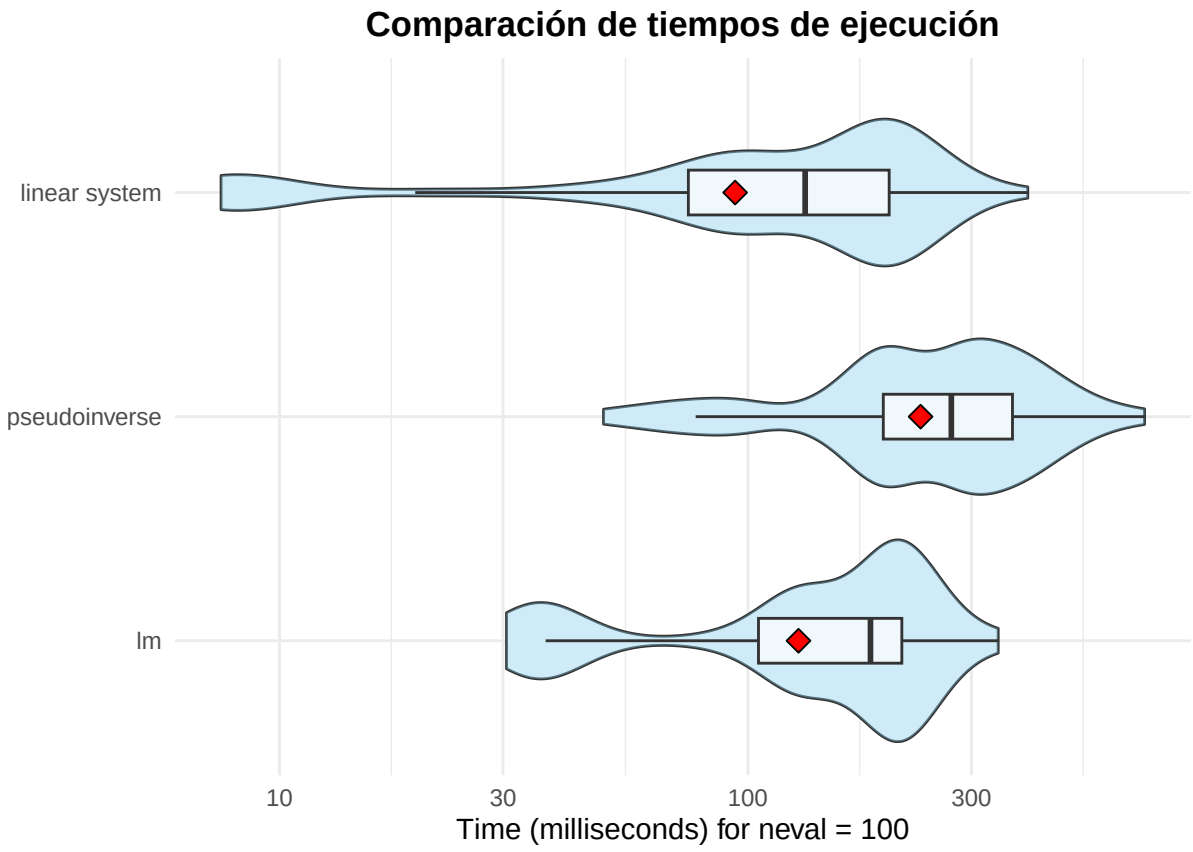
```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
##   Please report the issue at
```

```
##   <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



La línea: autoplot(mbm) genera automáticamente un gráfico a partir del objeto de microbenchmark, y por defecto ese gráfico es un boxplot (diagrama de cajas), que muestra: mediana, cuartiles, dispersión de los tiempos. No es un gráfico de violín. Entonces, este gráfico que sigue es mucho mejor que solo boxplot.El violín deja ver si hay:colas largas, multimodalidad y variabilidad real.

```r
library(ggplot2)
autoplot(mbm) +
geom_violin(fill = "skyblue", alpha = 0.4, color = NA) + # distribución
geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.7) + # resumen
stat_summary(fun = mean, geom = "point",
shape = 23, size = 3, fill = "red") + # media
labs(
title = "Comparación de tiempos de ejecución",
x = "Método",
y = "Tiempo (ms)"
) +
theme_minimal() +
theme(
legend.position = "none",
plot.title = element_text(hjust = 0.5, face = "bold")
)
```


Comparación de tiempos de ejecución
linear system
pseudoinverse
lm
10
30
100
300
Time (milliseconds) for neval = 100